



Task Master - Gestor de tareas personales.

1.- Reto 1.....	2
1.1.- Modelo de datos del sistema.....	2
1.2.- Diagrama entidad-relación.....	3
1.3.- Modelo relacional.....	4
1.4.- Estructura básica de la web.....	5
1.4.1.- index.html.....	5
1.4.2.- contacto.html.....	6
1.4.3.- index.css.....	7
1.4.4.- contacto.css.....	7
1.4.5.- comun.css.....	8
2.- Reto 2.....	10
2.1.- Diseño de las tablas.....	10
2.1.1.- USUARIO.....	10
2.1.2.- TAREA.....	10
2.1.3.- USUARIO - TAREA.....	11
2.1.4.- ESTADO.....	11
2.1.5.- CATEGORÍA.....	11
3.- Reto 3.....	12
3.1.- Modelado UML.....	12
3.2.- Diagrama de casos de uso.....	13
3.3.- Consultas SQL.....	14
3.3.1. Consulta para mostrar las tareas por estado.....	14
3.3.2. Consulta para mostrar las tareas completadas.....	15
4. Reto 4.....	16
4.1.- Revisión del modelado UML.....	16
4.2.- Revisión del diagrama de casos de uso.....	19
4.3.- Conclusiones.....	21
5.- Reto 5.....	22
5.1.- Inicio.....	22
5.2.- Gestión de usuarios.....	23
5.3.- Gestión de tareas.....	23
5.4.- Contacto.....	24
6.- Reto 6.....	25
6.1.- Casos de prueba y JUnit.....	25
6.2.- Code Smells.....	29
7.- Reto 7.....	30
8.- Repositorio y exposición.....	31

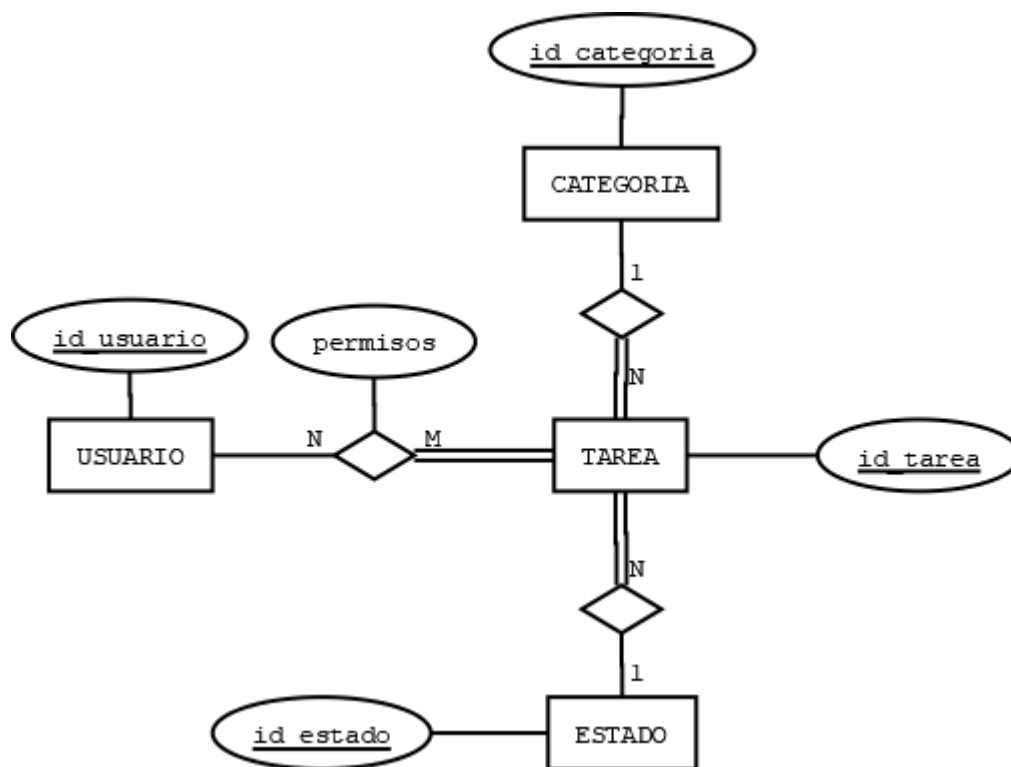
1.- Reto 1.

1.1.- Modelo de datos del sistema.

Para diseñar el modelo de datos del sistema me gustaría centrarme en un principio en las cuatro entidades básicas que la aplicación va a tener: Los usuarios, las tareas, las categorías y el estado.

- En primer lugar, las tareas van a ser el eje central de la aplicación, ya que esta va a ser la única entidad que se relaciona directamente con el resto de entidades. Además de estar relacionada con el resto de entidades, las tareas participarán de manera total en cada una de sus relaciones.
- Cada uno de sus usuarios tendrá acceso a sus tareas (o las tareas que tenga compartidas) y podrá asignarles una categoría y un estado siempre que tenga los permisos correspondientes.
- En cuanto a las categorías, estas se relacionan directamente con las tareas, pero el usuario debería de poder acceder a ellas tanto a través de las tareas, como directamente o como ayuda para buscar entre las tareas a las que tiene acceso.
- Finalmente, el estado también se relaciona directamente con la entidad tarea y, al igual que las categorías, el usuario debería de poder acceder de ambas formas a esta entidad.

1.2.- Diagrama entidad-relación.



1.3.- Modelo relacional.

USUARIO (id_usuario, nombre, email, contraseña)

CP: id_usuario

TAREA (id_tarea, id_estado, id_categoria, titulo, descripcion, fecha_c, fecha_f, observaciones)

CP: id_tarea

Caj: id_estado → ESTADO (id_estado)

VNN: id_estado

Caj: id_categoria → CATEGORIA (id_categoria)

VNN: id_categoria

USUARIO - TAREA (id_usuario, id_tarea, permisos)

CP: id_usuario, id_tarea

Caj: id_usuario → USUARIO (id_usuario)

Caj: id_tarea → TAREA (id_tarea)

ESTADO (id_estado, nombre_estado, descripcion)

CP: id_estado

CATEGORIA (id_categoria, nombre, descripcion)

CP: id_categoria

1.4.- Estructura básica de la web.

1.4.1.- index.html

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <title>TaskMaster - Inicio</title>
  <link rel="stylesheet" href="css/comun.css">
  <link rel="stylesheet" href="css/index.css">
</head>

<body>
  <header>
    <h1>TaskMaster - Gestión de tareas</h1>
    <nav>
      <a href="index.html">Inicio</a>
      <a href="html/contacto.html">Contacto</a>
    </nav>
  </header>
  <main>
    <section>
      <h2>Sobre el proyecto</h2>
      <p>
        TaskMaster es una aplicación de gestión de tareas personales que permite
        organizar tareas por categorías y estados. Además, se pretende que las tareas puedan ser
        compartidas entre diversos usuarios y que cada uno de estos pueda tener diversos permisos
        asignados en cada tarea.
      </p>
    </section>

    <section>
      <h2>El equipo</h2>
      <p>
        Proyecto desarrollado por José Vicente Sánchez Vargues, estudiante del
        curso 1º DAM en el instituto IES Camp de Morvedre.
      </p>
    </section>
  </main>
  <footer>
    <p>José Vicente Sánchez Vargues - 1º DAM</p>
  </footer>
</body>
</html>
```

1.4.2.- contacto.html

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <title>TaskMaster - Contacto</title>
  <link rel="stylesheet" href="../css/comun.css">
  <link rel="stylesheet" href="../css/contacto.css">
</head>

<body>
  <header>
    <h1>TaskMaster - Gestión de tareas</h1>
    <nav>
      <a href="../index.html">Inicio</a>
      <a href="contacto.html">Contacto</a>
    </nav>
  </header>
  <main>
    <form>
      <h2>Contacto</h2>
      <label>
        Nombre: <br>
        <input type="text" name="nombre" required>
      </label>

      <label>
        Email: <br>
        <input type="email" name="email" required>
      </label>

      <label>
        Mensaje: <br>
        <textarea name="mensaje" required></textarea>
      </label>

      <button type="submit">Enviar</button>
    </form>
  </main>
  <footer>
    <p>José Vicente Sánchez Vargues - 1º DAM</p>
  </footer>
</body>
</html>
```

1.4.3.- index.css

```
section {  
    background-color: white;  
    padding: 15px;  
    margin-bottom: 20px;  
    border-radius: 5px;  
}
```

1.4.4.- contacto.css

```
form {  
    background-color: white;  
    padding: 20px;  
    width: 95%;  
    border-radius: 5px;  
}  
  
label {  
    display: block;  
    margin-bottom: 10px;  
}  
  
input,  
textarea {  
    width: 40%;  
    padding: 5px;  
    margin-top: 5px;  
}  
  
button {  
    background-color: #2c3e50;  
    color: white;  
    border: none;  
    padding: 10px;  
    cursor: pointer;  
}  
  
button:hover {  
    background-color: #1a252f;  
}  
  
h2 {  
    color: #2c3e50;  
}
```

1.4.5.- comun.css

```
* {
    font-family: Arial, sans-serif;
}

html, body {
    height: 100%;
}

body {
    display: flex;
    flex-direction: column;
    margin: 0;
    background-color: #afa2a2;
}

header {
    background-color: #2c3e50;
    color: white;
    padding: 15px;
}

header h1 {
    margin: 0;
}

nav {
    margin-top: 10px;
}

nav a {
    color: white;
    text-decoration: none;
    margin-right: 15px;
}

nav a:hover {
    text-decoration: underline;
}

main {
    flex: 1;
    padding: 20px;
}

footer {
    background-color: #2c3e50;
    text-align: center;
```



```
color: white;  
}
```

2.- Reto 2.

2.1.- Diseño de las tablas.

2.1.1.- USUARIO.

Campo	Tipo	Nulo	Descripción
id_usuario	INT	NO	Identificador del usuario
nombre	VARCHAR	NO	Nombre del usuario
email	VARCHAR	NO	Correo del usuario
contraseña	VARCHAR	NO	Contraseña del usuario

2.1.2.- TAREA.

Campo	Tipo	Nulo	Descripción
id_tarea	INT	NO	Identificador de la tarea
titulo	VARCHAR	NO	Título de la tarea
descripción	VARCHAR	SI	Descripción de la tarea
fecha_c	DATE	NO	Fecha de creación
fecha_f	DATE	SI	Fecha límite
observaciones	VARCHAR	SI	Observaciones de la tarea
id_estado	INT	NO	Estado actual de la tarea
id_categoria	INT	NO	Categoría de la tarea

2.1.3.- USUARIO - TAREA.

Campo	Tipo	Nulo	Descripción
id_usuario	INT	NO	Usuario asociado
id_tarea	INT	NO	Tarea asociada
permisos	VARCHAR	NO	Permisos del usuario sobre la tarea

2.1.4.- ESTADO.

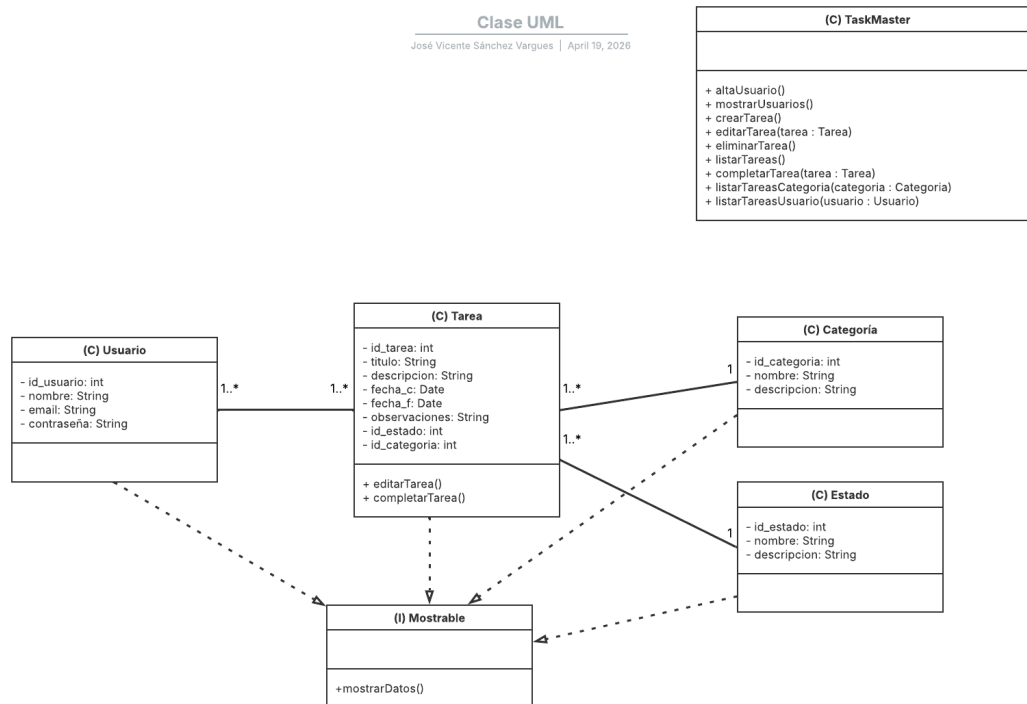
Campo	Tipo	Nulo	Descripción
id_estado	INT	NO	Identificador del estado
nombre_estado	VARCHAR	NO	Nombre del estado
descripcion	VARCHAR	SI	Descripción del estado

2.1.5.- CATEGORÍA.

Campo	Tipo	Nulo	Descripción
id_categoria	INT	NO	Identificador de la categoría
nombre	VARCHAR	NO	Nombre de la categoría
descripcion	VARCHAR	SI	Descripción de la categoría

3.- Reto 3.

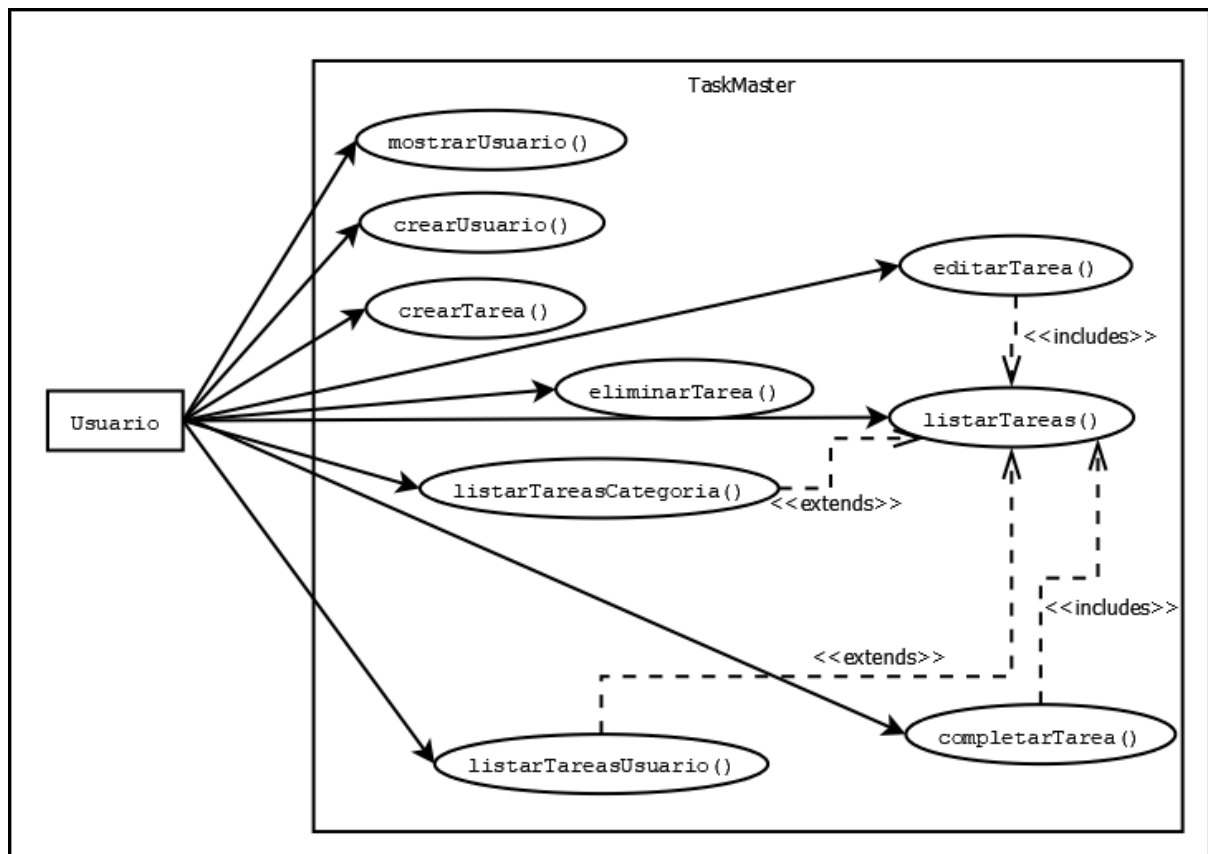
3.1.- Modelado UML.



Además de las clases Usuario, Tarea, Categoría y Estado, he decidido añadir al UML una clase principal llamada TaskMaster. Esta contiene la mayoría de métodos que utilizará el programa.

Por otro lado, a pesar de que por el momento únicamente necesitamos mostrar la información de los usuarios (en cuanto a tareas, por ejemplo, se nos pide únicamente listarlas) he decidido añadir una interfaz (Mostrable). Las cuatro clases implementarán esta interfaz con el objetivo de mostrar sus datos.

3.2.- Diagrama de casos de uso.



Además de los casos de uso básicos, en el esquema se pueden observar los siguientes <<includes>> y <<extends>>:

- **editarTarea() → listarTareas():** editarTarea() incluye listarTareas(), ya que podemos listar todas las tareas actuales para realizar la selección de la tarea que pretendemos editar.
- **completarTarea() → listarTareas():** editarTarea() incluye listarTareas(), ya que podemos listar todas las tareas actuales para realizar la selección de la tarea que pretendemos marcar como “Finalizada”.
- **listarTareasUsuario() → listarTareas():** listarTareasUsuario() extiende listarTareas(), ya que el proceso para listar las tareas sería el mismo, pero añadiendo la condición del usuario.
- **listarTareasCategoria() → listarTareas():** listarTareasCategoria() extiende listarTareas(), ya que el proceso para listar las tareas sería el mismo, pero añadiendo la condición de la categoría.

3.3. Consultas SQL.

3.3.1. Consulta para mostrar las tareas por estado.

En un principio, había planteado esta consulta de tal manera que mostrase todas las tareas actuales ordenadas por su estado:

```
SELECT *  
  FROM tarea  
 ORDER BY id_estado
```

Si se diese la situación en la que se quisiese seleccionar un estado específico y mostrar únicamente las tareas que se encuentran en este estado, ya que en esta quincena hemos empezado con las funciones en PL/pgSQL, he decidido añadir una variante que cumpla estos requisitos:

```
CREATE OR REPLACE FUNCTION seleccionEstado(seleccion VARCHAR) RETURNS int  
AS  
$$  
DECLARE  
  
BEGIN  
  IF seleccion = 'Pendiente'  
    THEN RETURN 1;  
  END IF;  
  IF seleccion = 'En progreso'  
    THEN RETURN 2;  
  END IF;  
  IF seleccion = 'Completada'  
    THEN RETURN 3;  
  END IF;  
  IF seleccion = 'Cancelada'  
    THEN RETURN 4;  
  END IF;  
END;  
$$  
language PLPGSQL;
```

```
SELECT *  
  FROM tarea  
 WHERE id_estado = seleccionEstado('En progreso')
```

3.3.2. Consulta para mostrar las tareas completadas.

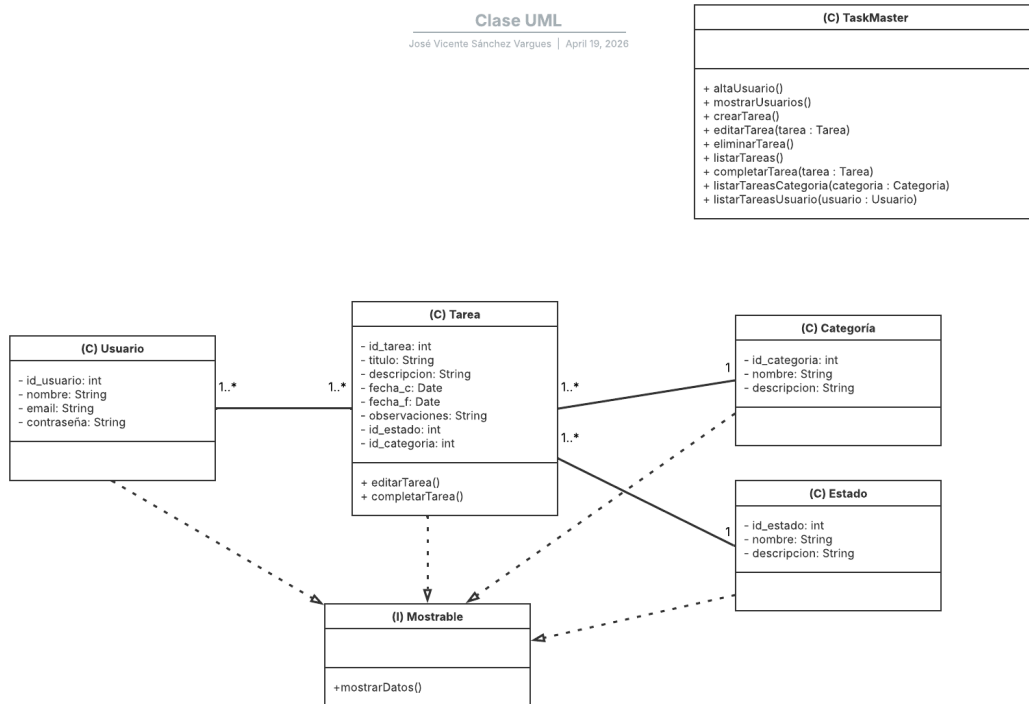
```
SELECT *  
  FROM tarea  
 WHERE id_estado = 3
```

Utilizamos “id_estado = 3” ya que este es la clave primaria que se le dió al estado ‘Completada’ al crear la base de datos.

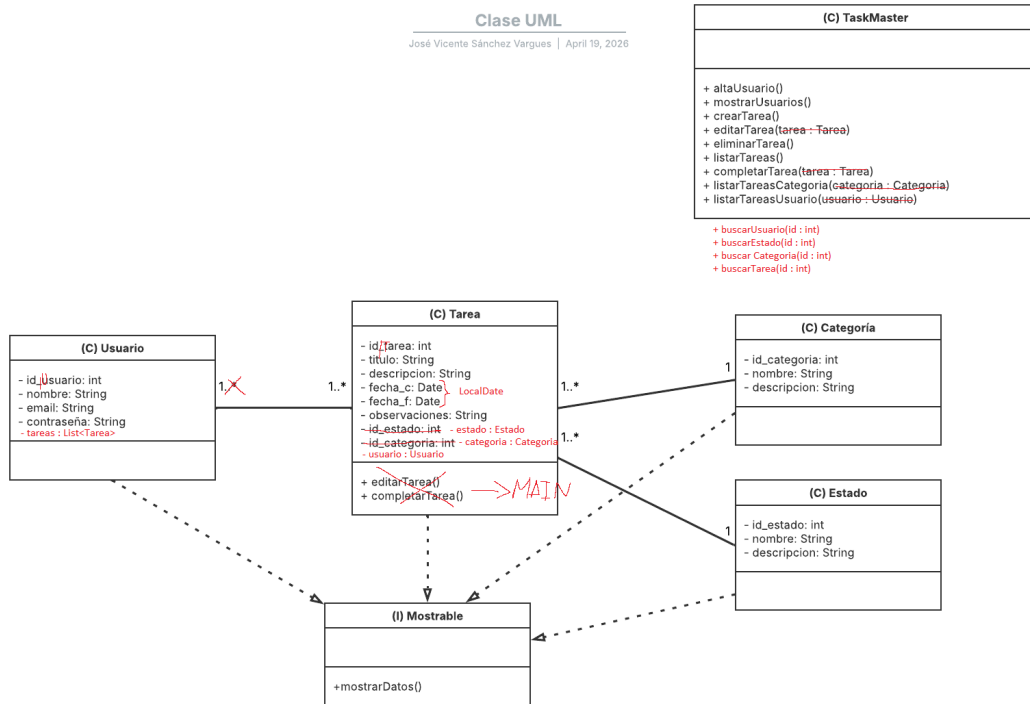
4. Reto 4.

4.1.- Revisión del modelado UML.

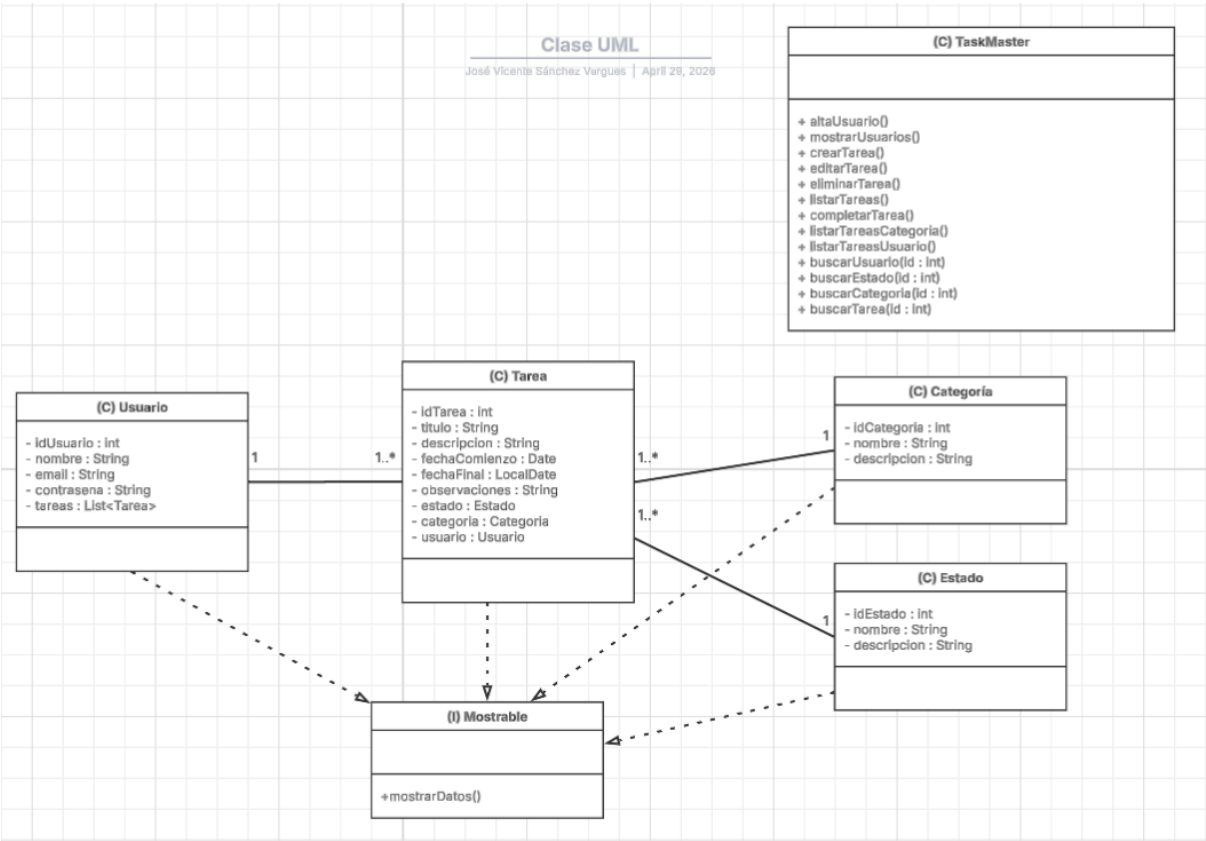
En primer lugar, tenemos el modelado UML que realicé durante la quincena pasada:



Durante la realización del código para esta actividad, me di cuenta de que el modelado UML que había propuesto en un principio contenía errores y se podía mejorar, por lo que fui marcando estos errores y mejoras en un nuevo archivo:

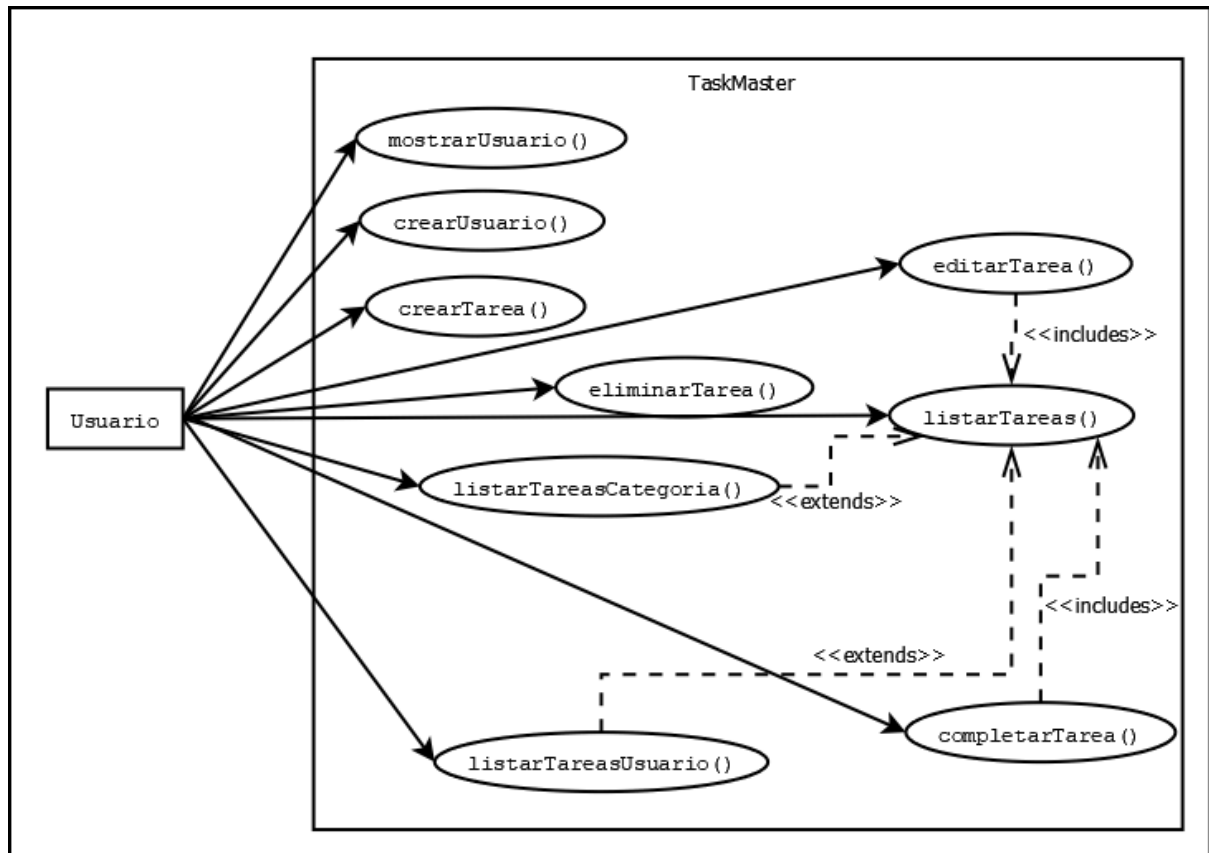


Finalmente, apliqué estas correcciones que había estado anotando en limpio para crear el modelado UML final:



4.2.- Revisión del diagrama de casos de uso.

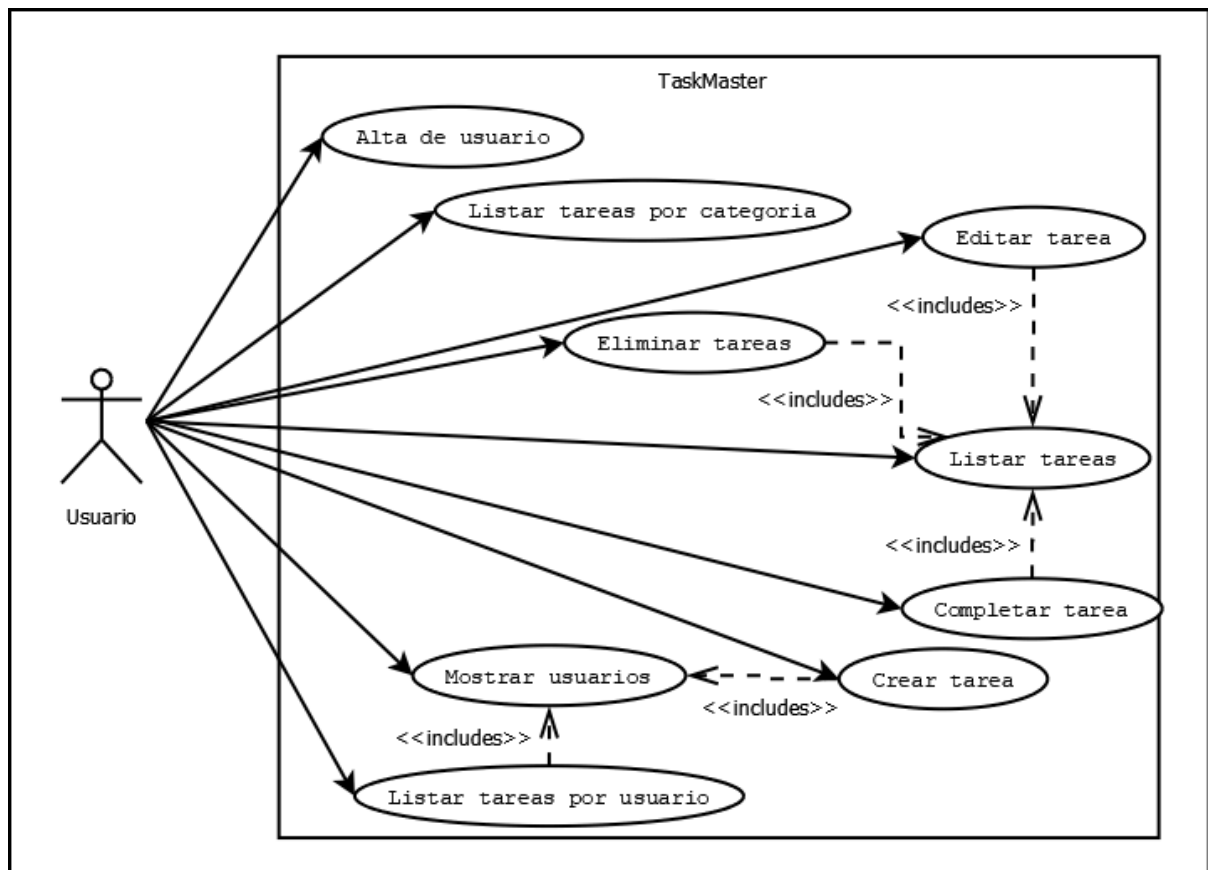
A continuación, tenemos el diagrama de casos de uso que propuse durante la quincena anterior:



Trás estudiar el temario de la asignatura “Entornos de Desarrollo” de esta quincena, aprendí a realizar un diagrama de casos de uso correctamente y modifiqué el anterior para ajustarlo al proyecto.

Durante estas modificaciones me percaté de que principalmente había malinterpretado el uso del `<<extends>>`, ya que este se debe usar para señalar que un caso de uso tiene un comportamiento opcional al ejecutar otro caso de uso.

Por otro lado, trás escribir el código del programa, me percaté de que habían muchos más casos de usos que incluían otros casos de uso. Este fue el resultado final trás la revisión y las modificaciones:



4.3. Conclusiones.

Durante el trabajo de esta quincena me he dado cuenta de la importancia de saber realizar un modelado UML y un diagrama de casos de uso correctamente, ya que al no tener mucha práctica en esto cometí varios errores realizando ambos esquemas durante la quincena pasada.

Con el objetivo de aprender a plantear mejor mis futuros proyectos, he decidido que trataré de poner más en práctica el diseño y uso de estos dos esquemas.

Finalmente, me gustaría añadir que, al ser esta la primera vez que he trabajado con un proyecto “grande” en GitHub, he aprendido a dividir estos proyectos en pequeñas tareas más cortas o sencillas de realizar, observar mis avances y progresos poco a poco y a organizarme de mejor manera.

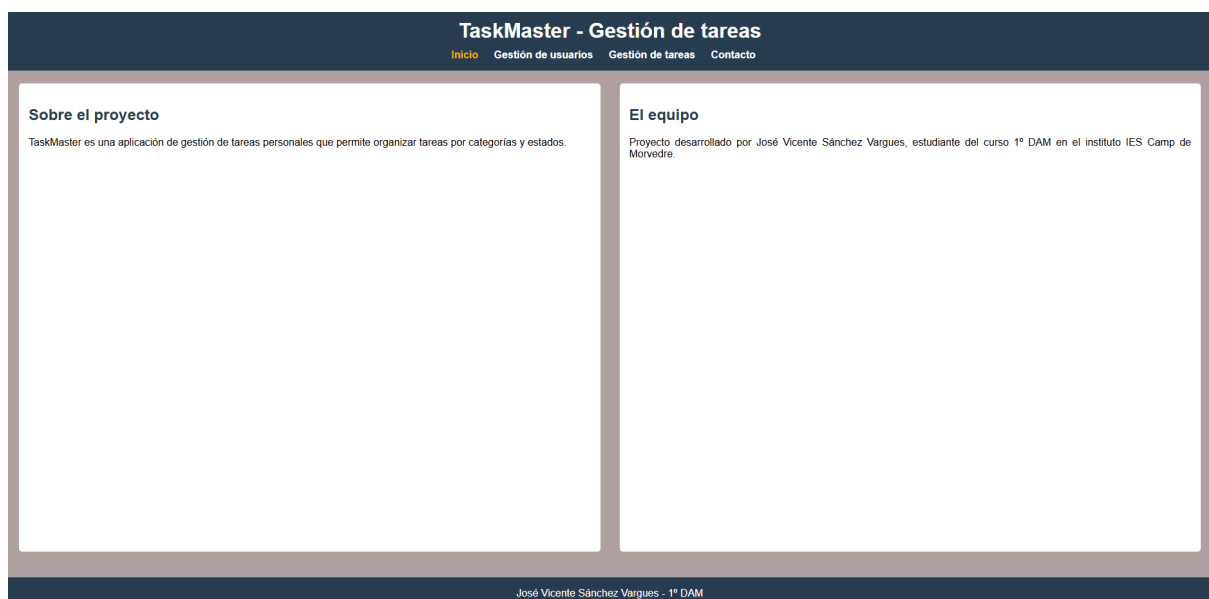
5.- Reto 5.

Durante la primera quincena de mayo realizamos el reto 5. Este reto consistía en realizar una propuesta de interfaz para la aplicación Task Master. Además, esta interfaz debía de presentar:

- Un mínimo de dos vistas.
- Cabecera y footer.
- Un formulario de contacto.
- Uso de Flexbox.
- Uso de, como mínimo, un propiedad de posicionamiento (absolute, relative o fixed).
- Un mínimo de una animación.
- Diseño responsive.

Adjunto a continuación capturas de pantalla de mi propuesta:

5.1.- Inicio.



5.2.- Gestión de usuarios.

TaskMaster - Gestión de tareas

[Inicio](#)[Gestión de usuarios](#)[Gestión de tareas](#)[Contacto](#)

[Alta de usuario](#)[Mostrar usuarios](#)

Alta de usuario

ID:

Nombre:

Email:

Contraseña:

Alta de usuario

José Vicente Sánchez Vargues - 1º DAM

5.3.- Gestión de tareas.

TaskMaster - Gestión de tareas

[Inicio](#)[Gestión de usuarios](#)[Gestión de tareas](#)[Contacto](#)

[Crear tarea](#)[Editar tarea](#)[Completar tarea](#)[Eliminar tarea](#)[Listar tareas](#)

Crear tarea

ID del usuario:

ID de la tarea:

Nombre de la tarea:

Descripción:

Fecha de inicio:

dd/mm/aaaa

Fecha de finalización:

dd/mm/aaaa

Estado de la tarea:

Categoría de la tarea:

Crear tarea

José Vicente Sánchez Vargues - 1º DAM

5.4.- Contacto.

TaskMaster - Gestión de tareas

[Inicio](#) [Gestión de usuarios](#) [Gestión de tareas](#) [Contacto](#)

Contacto

Nombre:

Email:

Mensaje:

Enviar

José Vicente Sánchez Vargues - 1º DAM

6.- Reto 6.

6.1.- Casos de prueba y JUnit.

ID	Caso de prueba	Resultado esperado
CP-01	Crear tarea válida	Tarea creada correctamente.
CP-02	Crear tarea con fecha de finalización anterior a fecha inicial.	Error. El programa señala que no es posible.
CP-03	Crear tarea para un usuario inexistente.	Error. El programa señala que se debe de usar un usuario existente
CP-04	Crear un usuario con alguno de los campos obligatorios (sin contar ID) vacíos.	Error. El programa señala que los campos son obligatorios.
CP-05	Editar una tarea inexistente.	Error. El programa señala que el ID ingresado no existe.
CP-06	Filtrar tareas de una categoría vacía.	El programa muestra una lista vacía.

```
package org.example.main;

import org.example.model.Categoria;
import org.example.model.Estado;
import org.example.model.Tarea;
import org.example.model.Usuario;
import org.junit.jupiter.api.Test;

import java.time.LocalDate;

import static org.junit.jupiter.api.Assertions.*;

class TaskMasterTest {

    @Test
    void crearTareaValida() {
        Usuario usuario = new Usuario(200, "Juan",
"juan@gmail.com", "1234");
        Categoria categoria = new Categoria(200, "Trabajo", "");
        Estado estado = new Estado(200, "Pendiente", "");
```

```
Tarea tarea = new Tarea(1, "Estudiar", "JUnit",
LocalDate.of(2026, 6, 11),
        LocalDate.of(2026, 6, 18), "", estado, categoria,
usuario);

assertNotNull(tarea);
assertEquals("Estudiar", tarea.getTitulo());
}

@Test
void fechaFinalAnteriorAInicial() {

    Usuario usuario = new Usuario(200, "Juan",
"juan@gmail.com", "1234");
    Categoria categoria = new Categoria(200, "Trabajo", "");
    Estado estado = new Estado(200, "Pendiente", "");

    Tarea tarea = new Tarea(1, "Estudiar", "JUnit",
LocalDate.of(2026, 6, 11),
        LocalDate.of(2026, 6, 18), "", estado, categoria,
usuario);

    assertNotNull(tarea);
    assertEquals(null, tarea);
}

@Test
void usuarioInexistente() {
    assertNull(TaskMaster.buscarUsuario(999));
}

@Test
void usuarioConCamposVacios() {

    Usuario usuario = new Usuario(200, "Juan",
"juan@gmail.com", "");
    assertEquals(null, usuario);
}

@Test
void editarTareaInexistente() {
    assertNull(TaskMaster.buscarTarea(999));
}

@Test
void categoriaVacía() {

    Categoria categoria = new Categoria(200, "Trabajo", "");
```

```

        boolean existeTarea = false;

        for (Object obj : TaskMaster.getTareas()) {
            Tarea tarea = (Tarea) obj;

            if (tarea.getCategoria().getId() == categoria.getId())
            {
                existeTarea = true;
                break;
            }
        }

        assertFalse(existeTarea);
    }
}

```

Los `@Test fechaFinalAnteriorAInicial()` y `usuarioConCamposVacios()` fallan, por lo que corregimos lo siguiente (esta corrección se realiza también en TaskMaster.java):

```

@Test
void usuarioConCamposVacios() {

    System.out.println("Ingrese el ID del usuario: ");
    int id = 200;

    Usuario usuario = null;

    if (buscarUsuario(id) == null){
        String nombre = "";

        String email = "";

        String contrasena = "";

        if (campoVacio(nombre) || campoVacio(email) ||
campoVacio(contrasena)){
            System.out.println("Por favor, rellene los campos
obligatorios para dar de alta a un usuario correctamente.");
        } else {
            usuario = new Usuario(id, nombre, email, contrasena);

            System.out.println("Usuario " + usuario.getId() + " - "
+ usuario.getNombre() + " dado de alta correctamente.");
            System.out.println();
        }
    }
    else {

```

```

        System.out.println("Ya existe un usuario con el ID " + id +
". Volviendo al menú principal.");
        System.out.println();
    }
    assertEquals(null, usuario);
}

```

Modificamos además el método `altaUsuario()` en `TaskMaster.java` y añadimos el siguiente método que comprobará si alguno de los campos están vacíos:

```

public static boolean campoVacio(String campo){
    if (campo.equals("")){
        return true;
    }
    return false;
}

```

Por otro lado, usamos el método `.isBefore()` de `LocalDate` para evitar que `fechaFinalAnteriorAInicial()` nos de error y ajustamos su uso en `crearTarea()` y `modificarTarea()`.

```

@Test
void fechaFinalAnteriorAInicial() {

    Usuario usuario = new Usuario(200, "Juan", "juan@gmail.com",
"1234");
    Categoria categoria = new Categoria(200, "Trabajo", "");
    Estado estado = new Estado(200, "Pendiente", "");
    Tarea tarea = null;

    LocalDate comienzo = LocalDate.of(2026, 6, 11);
    LocalDate finalizacion = LocalDate.of(2026, 6, 7);

    //https://www.geeksforgeeks.org/java/localdate-isbefore-method-in-java-with-examples/
    if (comienzo.isBefore(finalizacion) ||
comienzo.isEqual(finalizacion)) {
        tarea = new Tarea(1, "Estudiar", "JUnit", comienzo,
            finalizacion, "", estado, categoria, usuario);
    }

    assertNull(tarea);
}

```

6.2.- Code Smells.

Identificamos un **magic number** en el método `completarTarea()`

```
tarea.setEstado(buscarEstado(3));
```

Por lo que declaramos la siguiente variable final para corregirlo:

```
private static final Estado ESTADO_COMPLETADA = buscarEstado(3);
```

Por otro lado, encontramos en el código varias estructuras en las cuales realizamos **repeticiones de código**. Por ejemplo:

```
System.out.println("Ingrese el ID del usuario del que quiere  
listar las tareas: ");  
int id = sc.nextInt();  
  
if (buscarUsuario(id) != null){  
    Usuario usuario = buscarUsuario(id);
```

Por lo que lo corregimos de la siguiente manera:

```
System.out.println("Ingrese el ID del usuario del que quiere  
listar las tareas: ");  
Usuario usuario = buscarUsuario(sc.nextInt());  
sc.nextLine();  
  
if (usuario != null){
```

7.- Reto 7.

Riesgo	Factor de riesgo	Daños	Medida de prevención
Retraso en el desarrollo del proyecto.	Mala planificación de tareas o falta de coordinación entre los miembros del equipo.	No completar las funcionalidades antes de la fecha de entrega.	Establecer un calendario de trabajo con plazos a cumplir.
Pérdida del código fuente.	Borrado accidental de archivos o fallo del equipo informático.	Pérdida de trabajo realizado y necesidad de rehacer parte del proyecto o su totalidad.	Utilizar GitHub y realizar copias de seguridad periódicas.
Errores en la gestión de tareas.	Falta de validaciones en la aplicación.	Creación de tareas con datos inconsistentes.	Implementar pruebas unitarias y validaciones de entrada.

8.- Repositorio y exposición.

[Repositorio.](#)

[Exposición.](#)